

Session 08 — Study Guide: DTOs and Request Validation

Where the session ends

Both `ProjectsModule` and `WorkspacesModule` validate their incoming requests with real DTOs and a global `ValidationPipe`, instead of hand-written `if` statements. Both resources also gained a brand-new, validated `POST` (create) endpoint — neither could be created before this session, only read and updated.

The real gap this session closes

Session 07's `ProjectsController.update()` typed its request body as a plain inline object: `{ name?: string; key?: string }`. That type exists only while the code is being written — the moment it compiles to JavaScript, the type is gone completely. The ONLY thing that ever stopped bad data was two hand-written `if` statements, one per field, that someone had to remember to write. This session replaces that with something declared once and enforced automatically, everywhere.

What a DTO actually is

A DTO (Data Transfer Object) is a plain class describing the shape data must have when it crosses a real boundary — here, an HTTP request body. Nothing Nest-specific makes it a DTO by itself; it's a class, decorated field by field with `class-validator` decorators:

```
TS
export class UpdateProjectDto {
  @IsOptional()
  @IsString()
  @IsNotEmpty()
  name?: string;
}
```

Read a field's decorators left to right as a sentence: "optional, a string, not empty."

Why a class, and not an interface

A TypeScript `interface` / `type` is compile-time only — it is erased completely from the emitted JavaScript. Nothing is left at runtime for a decorator to attach to, and nothing for `class-transformer` to build a real `instanceof` from. A `class` is a real JavaScript construct that survives compilation — the only thing that makes the whole mechanism possible at all.

The ValidationPipe

One line in `main.ts`, registered once, applies to every route in the whole application:

TS

```
app.useGlobalPipes(new ValidationPipe({ whitelist: true }));
```

Before a controller method ever runs, `ValidationPipe` : turns the plain JSON body into a real instance of whichever DTO class the controller declared, runs every `class-validator` decorator on it, and — if anything failed — throws an automatic `400`, built from the real list of what went wrong. No `if` statement required. `whitelist: true` additionally strips any property the DTO doesn't declare, rather than passing it through untouched.

Validation vs. business rules

This is the single most important distinction this session teaches. "Is this a non-empty string?" is validation — a DTO's job, checked before the controller method even runs. "Is this key already used by a DIFFERENT project?" depends on the rest of the current data — no decorator can know that. It stays exactly where it already was: a real check, in the service, after the DTO's own checks already passed.

TS

```
update(id: number, dto: UpdateProjectDto): Project {
  const project = this.findOne(id);

  if (dto.key !== undefined) {
    const taken = this.projects.some(
      (candidate) => candidate.id !== id && candidate.key === dto.key,
    );
    if (taken) {
      throw new ConflictException(
        `key "${dto.key}" is already used by another project.`,
      );
    }
  }
  // ...
}
```

A real create endpoint, built with a DTO from the start

Projects/Workspaces never had a way to be created before this session — only read and updated.

`CreateProjectDto` requires both fields (no `@IsOptional()` at all — creating something genuinely needs both):

TS

```
export class CreateProjectDto {
  @IsString()
  @IsNotEmpty()
  name: string;

  @IsString()
  @IsNotEmpty()
  key: string;
}
```

TS

```
@Post()
create(@Body() dto: CreateProjectDto) {
  return this.projectsService.create(dto);
}
```

Common beginner mistakes

- **Typing `@Body()` as `any` or leaving the old inline object type.** `ValidationPipe` has nothing real to check against in either case — the request is silently accepted, with no error at all. The quietest, most dangerous mistake this session risks.
- **Forgetting `@IsOptional()` on an UPDATE dto's field.** Makes every `PATCH` require every field, even ones the caller only wanted to leave unchanged.
- **Forgetting to REMOVE `@IsOptional()` on a CREATE dto's field.** Makes it possible to "create" something with missing required data.
- **Forgetting to add a newly-used decorator to the `class-validator` import line.** A real mistake made (and caught by `tsc`) during this very session's own preparation — always check the import line first when a decorator name shows as undefined.
- **Deleting the uniqueness check along with the now-redundant manual empty-name check.** Only the shape check became redundant; the business-rule check never did.

Not yet

These exist in NestJS, but are deliberately **not** covered this session — do not use them yet, and do not worry if you have seen the names before:

- Guards, interceptors, custom exception filters
- Provider tokens, custom providers (`useClass` / `useValue` / `useFactory`), injection scopes
- Custom decorators (`createParamDecorator`)
- `@nestjs/config` / `.env` configuration
- Swagger/OpenAPI
- A deliberate, structured use of Nest's own `Logger`
- Any database (MongoDB, PostgreSQL, or otherwise)

- Authentication of any kind
- Next.js

Glossary

- **DTO (Data Transfer Object)** — a class describing the shape of data crossing a boundary.
- **Decorator** — `@Something`, attaches metadata (recap from Session 07 — `class-validator`'s decorators are the same mechanism, a different library).
- **Pipe** — runs between "the request arrived" and "the controller method runs," able to transform or reject data before any controller code executes.
- **Whitelist (in this context)** — "strip anything not explicitly declared on the DTO."
- **Validation** — checking the SHAPE of data (type, presence, emptiness).
- **Business rule** — a check that depends on the rest of the current data, not just one field's own value (e.g. uniqueness).

Review questions

1. Why does `ValidationPipe` have nothing to check if a controller's `@Body()` is still typed as `{ name?: string }` instead of a real DTO class?
2. Why does `UpdateProjectDto` use `@IsOptional()` on every field, but `CreateProjectDto` uses it on none?
3. Name the one check in `ProjectsService.update()` that `class-validator` could never express, and explain why.
4. What does `whitelist: true` do, and how is it different from `forbidNonWhitelisted: true`?

Answers

1. Plain object types and `any` are compile-time-only — there is no real class for `class-transformer` to build an instance of, so the pipe has nothing to run `class-validator` decorators against; it silently does nothing.
2. Updating something may only want to change one field at a time (the rest should stay as they are), so every field must be optional; creating something genuinely requires every required field to be present from the start.
3. The key-uniqueness check — it depends on comparing against every OTHER project currently in the list, not on the shape of the one incoming field, which is all any single decorator can ever see.
4. `whitelist: true` silently strips any undeclared property. `forbidNonWhitelisted: true` instead rejects the request with a `400` naming the exact unexpected property — and only has any effect when `whitelist` is also `true`.

What you should be able to explain now

- What a DTO is, and why it must be written as a class, not an interface.

- How `class-transformer`, `class-validator`, and `ValidationPipe` fit together, in one sentence each.
- The line between validation (a DTO's job) and a business rule (stays in the service), with a real example from this session's own code.
- Why a brand-new `POST` endpoint is built with its DTO from the very first line, not added after.